

SZEGEDI TUDOMANYEGYETEM

Informatikai Intezet

SZAKDOLGOZAT

TDLWebshop - épülelgepeszeti webshop es adminisztracios rendszer

Toth David Laszlo

Szeged, 2026

TDLWebshop - épületgépészeti webshop és adminisztrációs rendszer

Mezo	Tartalom
Készítette	Toth David Laszlo
Szak	[pontos szak megadása]
Intezet / tanszek	[intezményi követelmény szerinti pontos megnevezés]
Temavezető	Dr. Bilicki Vilmos, egyetemi docens
Hely és év	Szeged, 2026

Megjegyzés: a szögletes zárójelekben szereplő adatokat a végleges formai követelmények alapján kell kitölteni.

Feladatkiiras

A szakdolgozat feladata egy epulelgepeszeti termekeket kezelo webaruhaz es a hozza kapcsolodo adminisztracios rendszer megtervezese es megvalositasa. A rendszer celja, hogy egy kisebb vagy kozepes meretu szakmai kereskedes digitalis mukodeset tamogassa: a vesarloi oldalon termekbongeszes, kosar, rendelesleadas, profil es rendelestortenet erhető el, mig az admin oldalon termek-, keszlet-, rendeles-, felhasznalo- es helyszini vasarlaskezeles jelenik meg.

A munka soran kiemelt szempont volt, hogy a webshop ne csak latvanyos felulet legyen, hanem mernokileg is ertelmezhető MVP-kent mukodjon. Ennek resze a jogosultsagi modell, a Firestore biztonsagi szabalyok, a tranzakcios rendeleskezeles, a PDF bizonylat/szamla generalasa, a CSV alapu termekimport, valamint egy katalogushoz kotott AI asszisztens prototipusa is. A feladat hatara tudatosan nem egy teljes erteku, bankkartya-elfogadassal es vallalati ERP-integracioval rendelkezo eles webshop, hanem egy bizonyithatoan mukodo, szakdolgozati szinten bemutatathato webes rendszer.

Tartalmi összefoglaló

Szempont	Leiras
Tema megnevezese	TDLWebshop - epulelgepeszeti webshop es adminisztracios rendszer
Feladat	Egy olyan webes alkalmazas letrehozasa, amely vasarloi es adminisztracios folyamatokat is kezel.
Megoldasi mod	Angular alapu frontend, Firebase/Firestore adattarolas es jogosultsagkezeles, Cloudflare Worker alapu AI proxy.
Alkalmazott eszkozok	Angular, TypeScript, Firebase Authentication, Firestore, Firebase Hosting, Cloudflare Workers, OpenRouter API, GitHub Actions.
Elert eredmenyek	Mukodo vasarloi ut, admin felulet, rendeleskezeles, keszletfigyeles, CSV import, PDF bizonylat, AI asszisztens es dokumentalt teszteles.
Kulcsszavak	webshop, Angular, Firebase, Firestore, admin rendszer, epulelgepeszet, AI asszisztens, CI, MVP

Tartalomjegyzék

A végleges dokumentumban ezt Word automatikus tartalomjegyzekkel kell frissíteni.

Javaslat: Hivatkozások -> Tartalomjegyzék -> Automatikus tartalomjegyzék, majd teljes frissítés.

1. Bevezetes, problemafelvetes es celkituzes

Az online kereskedelembe a felhasználók ma már nem csak egy egyszerű terméklistát várnak el, hanem gyors keresést, egyértelmű készletinformációt, megbízható rendelési folyamatot és átlátható kommunikációt. Ez különösen igaz az épületgépészeti területre, ahol a termékek gyakran szakmai döntést igényelnek, a vásárló pedig nem mindig tudja pontosan, melyik alkatrész vagy berendezés illik a saját helyzetéhez.

A TDLWebshop alapötlete ebből a problémából indult ki. A cél egy olyan webáruház volt, amely nem általános fogyasztási cikkekre, hanem fűtéshez, vízszereleshez, hűtőtechnikához, szellőzéshez és szerelési anyagokhoz kapcsolódó termékekre épül. A rendszer egyszerre célozza a lakossági vásárlókat és azokat a szakmai vagy üzleti szereplőket, akik gyakran helyszíni vagy B2B jellegű vásárlást igényelnek.

A szakdolgozat nem csupán a frontend megjelenítésre összpontosít, hanem a teljesebb folyamatra: a termékadatok kezelésére, a kosár és checkout logikájára, a rendelések követésére, az adminisztrációs munkafolyamatokra, a jogosultságokra, a biztonsági szabályokra és a tesztelhetőségre. A dolgozat célja az is, hogy bemutassa, hogyan lehet egy hallgatói projektet termékszerű MVP irányába vinni.

A dolgozatban bemutatott rendszer fejlesztésénél fontos szempont volt a reprodukálhatóság. A repo tartalmazza a futtatási leírást, a környezeti változók mintáját, a CI ellenőrzést és a tesztelési dokumentációt. Ez azért lényeges, mert a szakdolgozat értékelése során nem elég az, hogy a rendszer egyszer már működött: bizonyítani kell, hogy más környezetben is értelmezhetően elindítható és ellenőrizhető.

1.1. A megoldando problema

Az épületgépészeti termékek kínálata sokféle kategóriát érint, és a termékek közötti különbség gyakran nem csak árban vagy márkában jelenik meg. Egy radiator, csaptelep, szivattyú, klíma vagy szellőztető esetében fontos lehet a méret, teljesítmény, alkalmazási terület, kompatibilitás és készlet. Emiatt a felhasználó számára különösen értékes, ha az oldal nem csak listáz, hanem segíti a választást.

A másik probléma az adminisztráció. Egy kisebb kereskedésnél a webes rendelések mellett előfordulhat helyszíni vásárlás, telefonos egyeztetés, mentett vásárló, céges kedvezmény, készletfigyelés és PDF bizonylat igény is. Ha ezek külön rendszerekben vagy kézzel vezetett táblázatokban jelennek meg, az hibát és adateltoreést okozhat.

A TDLWebshop ezeket a problémákat egy központi rendszerben kezeli. A vásárlói oldal a termékek megtalálására és a rendelés leadása felől közelít, az admin felület pedig a termékek, rendelések, készlet és vásárlók kezelésére. A két oldal közös adatmodellre épül, így ugyanaz a termék- és rendelésadat jelenik meg a különböző szerepkörök számára.

1.2. Celkituzes es MVP-hatar

A projekt célja nem egy teljes ipari webshoprendszer lemasolása volt, hanem egy olyan MVP létrehozása, amely a legfontosabb vásárlói és adminisztrációs folyamatokat bizonyíthatóan lefedi. Az MVP-határ meghatározása tudatos döntés volt, mert egy szakdolgozati projektben a túl sok felig kész funkció gyengítheti a rendszer megitelet.

1. tablázat - Az MVP-hatar osszefoglalasa

MVP resze	Rovid indoklas	Allapot
Termeklista es termekadatlap	A webshop alapja, a vasarloi tajekozodas kiindulopontja.	Megvalositva
Kosar es checkout	A vasarloi ut legfontosabb uzleti folyamata.	Megvalositva
Rendeleσκοvetes profilbol	Bizalmat es attekinthetoseget ad a vasarlonak.	Megvalositva
Admin termék- es készletkezelés	A bolt mukodtetesehez szukseges belso folyamat.	Megvalositva
Helyszini vasarlas	Domainhez illeszkedo B2B/uzleti kulonbseg.	Megvalositva
PDF bizonylat/szamla	Adminisztracios bizonyitek es szakdolgozati ertek.	Megvalositva
AI asszisztens	Katalogushoz kotott, korlatos szakmai seged.	Prototipus
Bankkartya-elfogadas	Valos penzugyi integracio magasabb kockazattal.	Tovabbfejlesztes
ERP integracio	Vallalati szintu illesztes, kulso rendszerekkel.	Tovabbfejlesztes

[KEP / ABRA HELYE] A TDLWebshop kezdolapja dark modban

Beillesztendo tartalom: Keszits kepernyokepet a fooldalrol ugy, hogy a kategoriak legordulo menuje nyitva van.

2. Piaci es területi összehasonlítás

A TDLWebshop pozicionalasahoz erdemes megvizsgalni, hogy egy epulelgepeszeti vagy muszaki webshop milyen funkciokat kinal a gyakorlatban. A piaci összehasonlitas celja nem az, hogy a projekt mindenben felulmulja a letezo rendszereket, hanem hogy megmutassa: a valasztott funkcioak valos területi igényekhez kapcsolodnak.

Az altalanos webshopok tobbnyire eros termeklistaval, kategoriakkal, keresovel es kosarral rendelkeznek, de a belso adminisztracios folyamatok vagy nem lathatok, vagy kulon vállalati rendszerben mukodnek. A szakdolgozat szempontjabol a TDLWebshop erteke eppen abban jelenik meg, hogy a vasarloi oldal mellett az admin oldal es a helyszini vasarlas is bemutatott resze a rendszernek.

A domain kulonlegessege, hogy az epulelgepeszeti termekeknek a vasarlo gyakran nem csak gyorsan rendelni akar, hanem szakmai dontesben is segitseget var. Emiatt a katalogushoz kotott AI asszisztens jo szakdolgozati kiegészites, de csak akkor, ha a korlatai vilagosak: nem helyettesiti a szakembert, es nem talalhat ki nem letezo termeket.

2. tablázat - Piaci funkciok összehasonlítása

Szepont	Altalanos webshop	Epulelgepeszeti webshop	TDLWebshop megoldasa
Termeklista	Altalanos kategoriak es keresesi lehetoseg.	Szakmai kategoriak, meretek, keszlet.	Futes, viz, hutes, szellozes, szerelési anyagok, lakossagi megoldasok.
Admin folyamat	Tobbnyire nem lathato a felhasznalónak.	Keszlet es rendeles gyors kezelese fontos.	Admin felulet, CSV import, keszletfigyeles, helyszini vasarlas.
Rendelezkovetes	Alap statuszok.	Fontos a keszlet es teljesites kovetese.	Profilbol lathato rendelesek es admin statuszvaltas.
B2B jelleg	Gyakran kulon rendszer.	Ceges kedvezmeny es mentett vasarlo fontos.	Mentett vasarlok, ceges jeloles, helyszini rogzitese.

AI támogatás	Ritkán katalógushoz kötött.	Szakmai kérdéseknél hasznos lehet.	Katalógus-alapu, korlátozott AI asszisztens OpenRouter proxyn keresztül.
--------------	-----------------------------	------------------------------------	--

2.1. A saját rendszer értéke

A TDLWebshop fő értéke nem egyetlen különleges funkcióban, hanem a folyamatok együttműködésében van. A vásárló termeket keres, kosárba tesz, rendelést ad le, majd a profiljában követi az állapotot. Az admin ugyanebben az adatkorben dolgozik: módosítja a rendelést, figyeli a készletet, PDF-et generál, illetve helyszíni vásárlást rögzít.

A rendszer oktatási szempontból azért is érdekes, mert több, egymással összefüggő technikai területet mutat be. Ilyen a kliensoldali Angular alkalmazás, a Firestore alapú adattárolás, a Firebase Authentication, a biztonsági szabályok, a CI ellenőrzés, valamint a szerveroldali titokkezeeléssel kialakított AI proxy.

A dolgozatban ezt az értéket úgy kell bizonyítani, hogy nem csak a kész képernyők látszanak, hanem az is, hogyan jut el az adat a felhasználói művelettől a Firestore dokumentumig, hogyan változik a rendelestörténet, és milyen ellenőrzések védik az adatokat.

[KEP / ABRA HELYE] Piaci összehasonlító táblázat

Beillesztendő tartalom: Ide beilleszthető a saját piackutatásból készült rövid táblázat vagy diagram.

3. Követelmények és use case-ek

A követelmények megfogalmazása azért fontos, mert a szakdolgozatban a rendszer működését nem elég felsorolni. Meg kell mutatni, hogy az egyes funkciók milyen felhasználói vagy adminisztrációs igényre adnak választ, és hogyan ellenőrizhető, hogy ezek ténylegesen működnek.

A TDLWebshop követelményei három nagy csoportba sorolhatók. Az első csoport a vásárlói folyamatokat tartalmazza: termékkeresés, kosár, checkout, profil és rendeléskövetés. A második az adminisztrációs folyamatokat fedi le: termékfeltöltés, rendeléskezelés, készlet, vásárló kezelés, kupon és PDF. A harmadik csoport a technikai és

minosegi kovetelmenyeket rogziti, peldaul a jogosultsagokat, biztonsagot, tesztelhetoseget es reprodukaltat.

3. tablazat - Kovetelmeny-traceability osszefoglalo

Azonosito	Kovetelmeny	Use case	Modul / bizonyitek
K1	A felhasznalo tudjon termeket keresni es kategoriak szerint bongeszni.	Termekkek bongeszese	Termeklista oldal, keresesi mezo, kategoriak
K2	A felhasznalo tudjon termeket kosarba tenni es mennyiseget modositani.	Kosar kezeles	CartService, kosar oldal
K3	A checkout ellenorizze az emailt, telefonszamot es kotelezo adatokat.	Rendeles leadasa	checkout.ts validacios logika
K4	A rendeles létrejotte utan az admin lassa es modositani tudja az allapotot.	Admin rendeleskezeles	admin.ts, order.service.ts
K5	A statuszvaltas audit es keszletvaltozas mellett tortenjen.	Rendeles teljesitese	OrderService tranzakcios logika
K6	A dolgozo csak korlatozott admin funkciokat erjen el.	Dolgozoi felulet	Firestore rules, admin jogosultsagi logika
K7	A PDF bizonylat tartalmazza a rendelest, vevot, tetelek es osszegeket.	Szamla / bizonylat letoltese	invoice.service.ts
K8	Az AI asszisztens ne talaljon ki termeket, csak katalogushoz kototten ajanljon.	AI kerdes megvalaszolasa	chatbot-llm.service.ts, Worker proxy

3.1. Fo use case-ek

4. tablázat - Use case-ek rovid osszefoglalasa

Use case	Nev	Leiras
UC1	Vasarlo termeket keres es kosarba teszi	A vasarlo kategoriat valaszt vagy keresoszot ad meg, megnyitja a termeket, majd kosarba helyezi.
UC2	Vasarlo rendelest ad le	A vasarlo kitolti a checkout urlapot, a rendszer validal, majd letrehozza a rendelest.
UC3	Admin teljesiti a rendelest	Az admin megnyitja a rendelest, ellenorzi az adatokat, statuszt modosit, a rendszer auditot es keszletvaltozast rogzit.
UC4	Admin helyszini vasarlast rogzit	Az admin mentett vasarlot valaszt vagy uj adatot ad meg, tetelet ad a vasarlashoz, majd PDF bizonylatot general.
UC5	Dolgozo termeket kezel	A dolgozo a szamara engedelyezett admin funkciókkal termeket tolthet fel es keszletet ellenorizhet.
UC6	Felhasznalo AI asszisztent hasznal	A felhasznalo katalogushoz vagy epulelgepeszethez kapcsolodo kerdest tesz fel, a rendszer korlatos valaszt ad.

3.1.1. Vasarlo termeket keres es kosarba teszi

A vasarlo kategoriat valaszt vagy keresoszot ad meg, megnyitja a termeket, majd kosarba helyezi.

Sikeres lefutas eseten a rendszer egyertelmu visszajelzest ad, az adat pedig a megfelelo Firestore gyujtemenyben vagy kliensoldali allapotban megjelenik. Hibaag eseten a felhasznalo validacios vagy jogosultsagi uzenetet kap, nem pedig csendes hibaval talalkozik.

3.1.2. Vasarló rendelest ad le

A vasarló kitölti a checkout urlapot, a rendszer validál, majd létrehozza a rendelest.

Sikeres lefutas esetén a rendszer egyértelmu visszajelzést ad, az adat pedig a megfelelő Firestore gyujtemenyben vagy kliensoldali állapotban megjelenik. Hibaag esetén a felhasználó validációs vagy jogosultsági üzenetet kap, nem pedig csendes hibával találkozik.

3.1.3. Admin teljesíti a rendelest

Az admin megnyitja a rendelest, ellenorzi az adatokat, státuszt modosit, a rendszer auditot és készletváltozást rögzít.

Sikeres lefutas esetén a rendszer egyértelmu visszajelzést ad, az adat pedig a megfelelő Firestore gyujtemenyben vagy kliensoldali állapotban megjelenik. Hibaag esetén a felhasználó validációs vagy jogosultsági üzenetet kap, nem pedig csendes hibával találkozik.

3.1.4. Admin helyszini vasarlast rögzít

Az admin mentett vasarlot választ vagy új adatot ad meg, tetelet ad a vasarlashoz, majd PDF bizonylatot generál.

Sikeres lefutas esetén a rendszer egyértelmu visszajelzést ad, az adat pedig a megfelelő Firestore gyujtemenyben vagy kliensoldali állapotban megjelenik. Hibaag esetén a felhasználó validációs vagy jogosultsági üzenetet kap, nem pedig csendes hibával találkozik.

3.1.5. Dolgozó termeket kezel

A dolgozó a számára engedélyezett admin funkciókkal termeket tolthat fel és készletet ellenorizhet.

Sikeres lefutas esetén a rendszer egyértelmu visszajelzést ad, az adat pedig a megfelelő Firestore gyujtemenyben vagy kliensoldali állapotban megjelenik. Hibaag esetén a felhasználó validációs vagy jogosultsági üzenetet kap, nem pedig csendes hibával találkozik.

3.1.6. Felhasználó AI asszisztent használ

A felhasználó katalogushoz vagy épütelgepeszethez kapcsolódó kérdést tesz fel, a rendszer korlatos választ ad.

Sikeres lefutas eseten a rendszer egyertelmu visszajelzest ad, az adat pedig a megfelelo Firestore gyujtemenyben vagy kliensoldali allapotban megjelenik. Hibaag eseten a felhasznalo validacios vagy jogosultsagi uzenetet kap, nem pedig csendes hibaval talalkozik.

[KEP / ABRA HELYE] Use case diagram

Beillesztendo tartalom: Ide illesztheto a vasarlo, admin, dolgozo es AI asszisztens kapcsolatát bemutato use case abra.

4. GUI/UX tervezes

A fejezetben a vegleges valtozatba valodi kepernyokepeket kell elhelyezni. A jelenlegi munkaverzio a kephelyeket es a kepalairasokat tartalmazza, hogy a konzulensi atnezesnel latszodjon a tervezett bizonyitasi logika.

A felulet tervezesenel a cel egy olyan modern, sotet alapu, ipari hangulatu arculat kialakitasa volt, amely illeszkedik az epulelgepeszeti temahoz. A dark mode alapertelmezett megjelenes, mert a TDLWebshop logoja es a technikai jellegu termekkor jól mukodik ebben a vizualis kornyezetben. A light mode ugyanakkor fontos kenyelmi es uzleti alternativa.

A design nem pusztan dekorativ szerepet tolt be. A felhasznalo folyamatokban a keresosav, kategoriak, kosar, profil, termekkartyak es CTA gombok elhelyezese azt szolgálja, hogy a vasarlo gyorsan eljusson a termekhez es a rendelesehez. Az admin felulet mas logikat kovet: ott a surubb, tablazatosabb es funkciokozpontu elrendezes a hatekonyabb.

A GUI/UX fejezetben a vegleges dolgozatba tenyleges kepernyokepeket kell beszurni. Ezek nem csak illusztraciok, hanem bizonyitekok arra, hogy a rendszer nem egy izolalt kodreszlet, hanem hasznalhato felhasznalo felulettel rendelkezo webalkalmazas.

5. tablazat - GUI/UX kepernyok es bizonyitekok

Kepernyo	Cel	Fontos allapot	Bizonyitek
Kezdolap	Első benyomas, kategoria es akcio kiemeles.	Dark mode, kategoriak dropdown.	Kepernyokep
Termeklista	Bongeszes, kereses,	Tobb termek, szurok,	Kepernyokep

	szures.	akcios termekek.	
Termekadatlap	Reszletes termékadatok es kosarba helyezés.	Kepgaleria, ar, készlet.	Kepernyokep
Kosar	Mennyiség, összeg, tovabblepes.	Tobb termék, torles es mennyiség.	Kepernyokep
Checkout	Adatbekeres es validacio.	Hibas email/telefon, sikeres rendeles.	Kepernyokep
Profil	Rendelestortenet es adatok.	Korabbi rendelések, statusz.	Kepernyokep
Admin	Belso folyamatok kezelese.	Termekimport, rendelések, készlet.	Kepernyokep
AI asszisztens	Katalogushoz kotott segitseg.	Domain kerdes es valasz.	Kepernyokep

[KEP / ABRA HELYE] Kezdolap dark mode, kategoriak legordulo menuvel

Beillesztendo tartalom: Fooldal, kategoriak menu nyitva.

[KEP / ABRA HELYE] Kezdolap AI asszisztenssel

Beillesztendo tartalom: Fooldal, AI ablak nyitva, egy relevans epulelgepeszeti kerdessel.

[KEP / ABRA HELYE] Termeklista szures/kereses kozben

Beillesztendo tartalom: Termekek oldal, keresoszo vagy categoria szuro aktiv.

[KEP / ABRA HELYE] Termekadatlap

Beillesztendo tartalom: Egy konkret termék oldala, termékkep, ar, készlet, kosar gomb lathato.

[KEP / ABRA HELYE] Kosar tobb termekkel

Beillesztendo tartalom: Kosar oldal legalabb ket tetellel.

[KEP / ABRA HELYE] Checkout validacio

Beillesztendo tartalom: Hibas email vagy telefonszam peldaval.

[KEP / ABRA HELYE] Checkout sikeres rendeles

Beillesztendo tartalom: Sikeres leadas utani visszajelzes.

[KEP / ABRA HELYE] Profil es rendeleskovetes

Beillesztendo tartalom: Vasarloi profil oldal korabbi rendelessel.

[KEP / ABRA HELYE] Kivansaglista

Beillesztendo tartalom: Kivansaglista oldal mentett termekkel.

[KEP / ABRA HELYE] Admin attekintes

Beillesztendo tartalom: Admin fo nezet statisztikai kartyakkal.

[KEP / ABRA HELYE] Admin termekkezeles es CSV import

Beillesztendo tartalom: Admin termekek ful CSV import resszel.

[KEP / ABRA HELYE] Admin keszletfigyeles

Beillesztendo tartalom: Keszlet ful alacsony keszlet jelzessel.

[KEP / ABRA HELYE] Helyszini vasarlas

Beillesztendo tartalom: Mentett vasarlo kivalasztva, tetelek hozzaadva.

[KEP / ABRA HELYE] PDF szamla vagy bizonylat

Beillesztendo tartalom: General PDF megnyitva, fejléc es tetelsorok lathatok.

[KEP / ABRA HELYE] Admin felhasznalo es jogosultsag kezeles

Beillesztendo tartalom: Felhasznalok ful admin/dolgozo szerepkorokkal.

[KEP / ABRA HELYE] GitHub Actions zold CI

Beillesztendo tartalom: GitHub Actions lista legfrissebb zold futással.

5. Technologiai hatter

A TDLWebshop Angular alapu egyoldalas webalkalmazaskent keszult. Az Angular valasztasa azert indokolt, mert komponensalapu felépítést, eros TypeScript tamogatást es strukturált szolgáltatásreteget ad. Egy webshop es admin rendszer eseteben ez különösen hasznos, mert a terméklista, kosar, checkout, profil es admin felület külön komponensekre bontható.

Az adattárolási es hitelesítési reteget Firebase szolgáltatások adják. A Firestore dokumentumalapú modellje jól illeszkedik a termékek, rendelések, felhasználói profilok,

kuponok es auditbejegyzesek tarolasahoz. A Firebase Authentication segitsegevel a bejelentkezés es szerepkorhoz kotott feluletkezeles is egyszerubben megvalosithato.

A rendszerben megjelenő AI asszisztens külső LLM-szolgáltatásra épül, de a kliens nem közvetlenül hívja az OpenRouter API-t. A kulcs védelme érdekében egy Cloudflare Worker proxy kezeli a szerveroldali hívásokat. Ez fontos biztonsági döntés, mert API kulcsot nem szabad nyilvános frontend kódba egetni.

6. táblázat - A rendszerben használt technológiák

Technologia	Szerep a rendszerben	Indoklas
Angular	Frontend keretrendszer	Komponensalapú, TypeScript alapú, nagyobb felülethez jól strukturált.
TypeScript	Alkalmazaslogika	Tipusosság, attekinthetőbb szolgáltatások es komponensek.
Firebase Authentication	Felhasználóazonosítás	Emailés beépés, szerepkorhoz kóthető profilok.
Cloud Firestore	Adattárolás	Dokumentumalapú modell, realis webshop entitasokhoz illeszthető.
Firestore Rules	Biztonsági retegek	Jogosultságok es payload validacio adatbázis szinten.
Firebase Hosting	Publikálás	Egyszerű deploy es webes elérés.
Cloudflare Worker	AI proxy	API kulcs védelme es CORS kezeles.
GitHub Actions	CI ellenorzés	Build es teszt futtatás bizonyitása.

5.1. Alternatívák es döntések

A backend megvalósítása történhetett volna saját Node.js/Express szerverrel is. A Firebase választása azért volt kedvező, mert a szakdolgozati célhoz gyorsabban adott hitelesítést, hostingot, adatbázist es biztonsági szabályokat. Ez nem jelenti azt, hogy minden üzleti logika ideálisan kliensoldalon maradhat; a dolgotban ezt MVP-korlatként is rögzíteni kell.

A bankkartya-elfogadás és teljes pénzügyi integráció tudatosan kimaradt. Ennek oka, hogy egy valószínű fizetési szolgáltató beiktatása jogi, biztonsági és tesztelési követelményeket hozna be. A rendszerben a fizetési módok adminisztratíván jelennek meg, de a valószínű pénzügyi mozgást a projekt nem kezeli.

Az AI asszisztens esetében szintén fontos volt a határhúzás. A cél nem egy teljes szakmai tanácsadó rendszer, hanem egy katalógushoz kötött, segítő jellegű funkció. Ha a kérdés túl távoli vagy nincs pontos terméktalálat, a rendszernek nem szabad magabiztosan kitalált terméket ajánlania.

6. Architektúra és adatáramlás

A rendszer architektúrája három fő rétegre bontható. Az első a kliensoldali Angular alkalmazás, amely a felhasználói felületet, a komponenseket és a szolgáltatásokat tartalmazza. A második a Firebase rétege, ahol a hitelesítés, Firestore adatbázis, biztonsági szabályok és hosting jelenik meg. A harmadik a külső integrációs réteg, amelybe az AI proxy és az OpenRouter szolgáltatás tartozik.

Az Angular alkalmazásban a komponensek a megjelenítésért és felhasználói interakcióért felelnek, míg a szolgáltatások kezelik az adatműveleteket. Ilyen szolgáltatás a kosár, rendelés, számla, profil, AI asszisztens vagy termékkezelés logikája. Ez a szétválasztás segít abban, hogy a felület ne közvetlenül tartalmazza az összes üzleti logikát.

Az adatáramlás példája a checkout folyamaton jól bemutatható. A vásárló a felületen megadja az adatokat, a komponens validál, majd a rendelési szolgáltatás létrehozza a Firestore dokumentumot. Az admin felület később ugyanezt a rendelest olvassa, státuszt módosít, auditbejegyzést készít és bizonyos esetekben készletváltozást hajt végre.

[KEP / ABRA HELYE] Komponens-architektúra ábra

Beillesztendő tartalom: Angular frontend, Firebase/Firestore, Firestore Rules, Cloudflare Worker és OpenRouter kapcsolatainak bemutatása.

[KEP / ABRA HELYE] Checkout szekvencia ábra

Beillesztendő tartalom: Vásárló -> Angular checkout -> OrderService -> Firestore -> admin rendeléslista folyamat.

6.1. Modulok

7. táblázat - Fő modulok és kapcsolataik

Modul	Feladat	Kapcsolodo adat
Home / Product UI	Kezdolap, kategoriak, termekkartyak.	products
Cart	Kosar állapot, mennyiség, osszegzes.	local state, products
Checkout	Adatbekeres, validacio, rendelés létrehozása.	orders
Profile	Felhasználói adatok és rendelések.	users, orders
Admin	Termék, rendelés, készlet és vásárló kezelése.	products, orders, savedCustomers
Invoice	PDF bizonylat generalása.	orders, order items
AI assistant	Katalogushoz kötött válaszadás.	products, Worker proxy

[KODRESZLET HELYE] `src/pages/checkout/checkout.ts` - kb. 367. sortol

Mi legyen látható a kódreszletben: A rendelés véglegesítésének folyamata, validáció és adatösszeállítás.

[KODRESZLET HELYE] `src/app/services/order.service.ts` - 41-127. sor

Mi legyen látható a kódreszletben: Statusz, audit és készletváltozás tranzakciós kezelése.

[KODRESZLET HELYE] `workers/openrouter-proxy/src/index.js` - teljes proxy lényegi része

Mi legyen látható a kódreszletben: OpenRouter hívás szerveroldali kulcskezeléssel és CORS korlátozással.

7. Adatmodell

A Firestore dokumentumalapú adatmodellje nem relációs táblakból, hanem gyűjteményekből és dokumentumokból épül fel. Ez gyors fejlesztést tesz lehetővé, de megköveteli, hogy az entitások és mezők tudatosan legyenek meghatározva. A TDL Webshop esetében a legfontosabb entitások a termék, kosartétel, rendelés, felhasználói profil, mentett vásárló, kupon, számla és auditbejegyzés.

Az adatmodell tervezésénél fontos szempont volt, hogy a vásárlói és admin felület ugyanarra az adatbázisra épüljön. A termékeket a vásárló listaként látja, az admin pedig szerkesztheti, importálhatja vagy készlet szempontból vizsgálhatja. A rendeléseket a vásárló saját profiljából követi, az admin pedig státuszt és teljesítést kezel rajtuk.

A dokumentumalapú modell egyik kockázata, hogy a túl laza szerkezet adatminőségi problémákhoz vezethet. Emiatt a Firestore rules és az alkalmazás oldali validáció különösen fontos: nem elég az adatot elmenteni, azt is ellenőrizni kell, hogy a mezők megfelelő típusúak és jogosultsággal írhatók.

8. táblázat - Adatmodell fő entitásai

Entitas / gyujtemeny	Fontos mezok	Kapcsolat / szerep
products	name, sku, category, price, stock, images, discount	Termeklista, admin termékkezelés, AI katalogus.
orders	customer, items, total, status, paymentMethod, createdAt	Checkout és admin rendeléskezelés.
orderStatusAudit	orderId, oldStatus, newStatus, actor, createdAt	Statuszváltozás nyomon követése.
users	email, role, disabled, profile data	Jogosultság és profiladatok.
savedCustomers	name, email, phone, company, taxNumber, disabled	Helyszíni vásárlás és mentett vásárló kezelése.
coupons	code, discount, active, validity	Kedvezmény logika.
newsletter	email, createdAt	Hírlevél feliratkozás admin oldali láthatósága.

[KEP / ABRA HELYE] Adatmodell diagram

Beillesztendő tartalom: Mermaid vagy PlantUML diagram a products, orders, users, savedCustomers, coupons és audit kapcsolataival.

[KODRESZLET HELYE] src/app/services/order.service.ts - 269-302. sor

Mi legyen látható a kódreszletben: Számlaszám általános logikája és rendeléshez kapcsolása.

8. Megvalositas kulcsfolyamatai

A megvalositas fejezet celja, hogy ne csak a hasznalt technologiakat sorolja fel, hanem bemutassa a rendszer legfontosabb mukodesi pontjait. A TDLWebshop eseteben ilyen kulcsfolyamat a checkout, a rendelések statuszkezelese, a helyszini vasarlas, a PDF bizonylat, a CSV termekimport, a jogosultsagkezeles es az AI asszisztens.

A checkout folyamatban a felhasznalo adatai, a kosar tartalma, a szallitasi es fizetesi mod, valamint az esetleges kupon egy kozos rendelési objektumma áll össze. A rendszer validálja a kotelezo mezoket, majd a rendeles létrehozasaival a vasarloi ut lezárul. A sikeres rendelés után a vasarlo visszajelzest kap, az admin pedig látja a rendelest.

A rendeles statuszkezelesnel fontos mernoki szempont, hogy a statuszvaltas ne elszigetelt mezomodositas legyen. A rendszer auditbejegyzest keszit, es bizonyos esetekben keszletvaltozast is vegrehajt. Ez a logika a dolgozatban kulon hangsulyozhato, mert megmutatja, hogy a rendszer az üzleti kovetkezményeket is figyelembe veszi.

8.1. Checkout es rendelesmentes

A checkout a vasarloi oldal legkritikusabb folyamatainak egyike. Itt derul ki, hogy a termekbongeszes es kosar logika valos rendelese alakul-e. A felületen a felhasznalo megadja a szamlazasi, szallitasi es kapcsolattartasi adatokat, a rendszer pedig ellenorzi az email es telefonszam formatumat is.

MVP-korlatkent fontos megemliteni, hogy a webes rendelés eseteben bizonyos ár- es kedvezményadatok kliensoldalrol érkeznek. Ez szakdolgozati prototipusnal elfogadhatóan dokumentálható, de eles webshopban szerveroldali újraszámolásra lenne szükség. A dolgozatban ezt nem elrejtteni kell, hanem mernoki tudatossaggal leírni.

[KODRESZLET HELYE] src/pages/checkout/checkout.ts - kb. 367. sortol

Mi legyen látható a kodreszletben: Rendelés veglegesítése, checkout adatok összeallitása es hibaagak.

8.2. Statusz, audit es keszlet

A rendelések teljesítése es torlese keszletoldali kovetkezményekkel járhat. A helyes megoldás celja, hogy a rendeles allapota, az auditbejegyzes es a keszletlogika osszhangban legyen. Ez szakdolgozati szempontbol az egyik legerosebb resz, mert nem pusztan felületi CRUD muveletrol van szo.

[KODRESZLET HELYE] src/app/services/order.service.ts - 41-127. sor

Mi legyen lathato a kodreszletben: Statuszvaltas, auditbejegyzes es keszlet tranzakcios kezelese.

8.3. Helyszini vasarlas

A helyszini vasarlas funkcio a rendszer domainhez illeszkedo kulonlegessege. Az admin vagy dolgozo mentett vasarlot valaszthat, uj adatokat adhat meg, termeket kereshet, mennyiseget allithat, majd a vasarlas utan PDF bizonylatot tolthet le. Ez a folyamat a B2B es boltban torteno ertekesitesi helyzeteket modellezi.

[KODRESZLET HELYE] src/app/services/order.service.ts - 222-267. sor

Mi legyen lathato a kodreszletben: Helyszini rendeles tranzakcios mentese es termektetelek kezelese.

8.4. CSV import

A termekfeltoltesnel a kezi rogzitest kiegesziti a CSV import. Ez kulonosen akkor hasznos, ha tobb tucatszer termeket kell feltolteni kepekkel, arakkal, SKU-val es kategoriakkal. A validacios lepes azert fontos, mert importnal egy hibas oszlop vagy elvalasztott karakter sok dokumentumot ronthatna el.

[KODRESZLET HELYE] src/pages/admin/admin.ts - 1181-1257. sor

Mi legyen lathato a kodreszletben: CSV import validacio, hibas sorok jelzese es mentheto termek feldolgozasa.

8.5. PDF bizonylat

A PDF bizonylat/szamla generalasa az admin folyamatok egyik lathato eredménye. A dokumentum tartalmazza a kiallito es vevo adatait, a rendeles azonositojat, a tetelek listajat, a netto es brutto osszegeket, valamint a fizetesi es szallitasi adatokat.

[KODRESZLET HELYE] src/app/services/invoice.service.ts - 9-154. sor

Mi legyen lathato a kodreszletben: PDF szamla felépítése, fejléc, vevo/kiallito blokkok, tetelsorok es osszesites.

[KEP / ABRA HELYE] General PDF bizonylat

Beillesztendo tartalom: A PDF-et nyisd meg es keszits kepernyokepet ugy, hogy a fejléc, vevo es tetelsorok latszodjanak.

8.6. AI asszisztens

Az AI asszisztens a saját termékkatalogusból kiindulva próbál segítséget adni. A cél az, hogy a válasz ne legyen független a webshop adataitól. Ha nincs pontos találat, a rendszernek inkább általános szakmai irányt kell adnia, és jeleznie kell, hogy pontos ajánlathoz emailés vagy személyes egyeztetés szükséges.

[KODRESZLET HELYE] src/app/services/chatbot-llm.service.ts - 26-84. és 217-236. sor

Mi legyen látható a kódreszletben: AI domainkorlátozás, kataloguslogika és nem releváns kérdések kezelése.

9. Biztonság és adatvédelem

A webes rendszerekben a biztonság nem kiegészítő elem, hanem alapkövetelmény. A TDLWebshop esetében különösen fontos a felhasználói adatok, rendelési adatok, admin jogosultságok, kuponok, PDF-ek és AI API kulcsok kezelése. A dolgozatban ezeket nem elég megemlíteni: be kell mutatni, milyen kockázatot jelentenek és milyen megoldás csökkenti a kockázatot.

A Firebase/Firestore használata miatt a biztonsági szabályok kulcsszerepet kapnak. A frontend kód önmagában nem véd elegendő, mert egy rosszindulatú kliens közvetlenül is próbálkozhat adatbázis-műveletekkel. Emiatt a Firestore rules feladata, hogy szerepkor, aktív felhasználó és payload szerkezet alapján korlátozza az olvasást és írást.

A titokkezelés szempontjából külön fontos, hogy API kulcs vagy jelszó ne kerüljön a repositoryba. Az OpenRouter kulcs a Cloudflare Worker secretjeként tárolando, a repóban pedig csak .env.example és dokumentált konfigurációs változók szerepelhetnek.

9. táblázat - Biztonsági minimum és kockázatok

Kockázat	Leírás	Kezelés a projektben	További javaslat
Jogosulatlan admin művelet	Normal user admin adatot módosítana.	Szerepkor alapú rules és frontend guard.	Custom claim és token revocation és rendszerben.
Tiltott felhasználó	Disabled user tovább próbálkozik.	Aktív user ellenőrzés szabályokban.	Auth szintű tiltás és rendszerben.

Kupon visszaeles	Kliensoldali kuponmanipulacio.	Kuponvalidacio es dokumentalt MVP-korlat.	Szerveroldali ujraszamolas.
PDF adatvedelem	Szemelyes adatok szerepelnek a bizonylaton.	Csak jogosult feluleten generalhato.	Letoltesi naplozas es tarolasi szabaly.
AI API kulcs kiszivargas	Kulcs frontendbe vagy gitbe kerul.	Cloudflare Worker secret, .env.example.	Rate limit es kvota.
Vendeg rendelés azonositas	Email-alapu osszekotes kockazata.	Dokumentalt korlat.	Regisztralt fiokhoz kotott rendelestortenet erosítése.

[KODRESZLET HELYE] firestore.rules - 25-76. sor

Mi legyen lathato a kodreszletben: Aktiv felhasznalo, admin es dolgozo jogosultsag ellenorzese.

[KODRESZLET HELYE] firestore.rules - 294-361. sor

Mi legyen lathato a kodreszletben: products, orders, users, savedCustomers es audit szabalyok.

[KODRESZLET HELYE] src/pages/admin/admin.ts - 607-748. sor

Mi legyen lathato a kodreszletben: Admin es dolgozoi jogosultsagok feluleti kezelese.

9.1. MVP-korlatok biztonsagi szempontbol

A szakdolgozati rendszer nem vállalja egy teljes erteku penzugyi backend szerepet. A bankkartya elfogadas, szerveroldali arujraszamolas, keszletfoglalas es ERP-szintu integracio tovaabfejlesztési irány. Fontos, hogy ez nem hiba, ha a dolgozatban vilagosan szerepel mint tudatos MVP-hatar.

Az AI asszisztensnel a CORS beallitas onmagaban nem teljes visszaeles elleni vedelem. Ezt rate limit, kvota vagy felhasznaloi azonositashoz kotott hivaslimit erositheti. A dolgozatban ezt a reszt ugy erdemes bemutatni, mint mukodo prototipust, amelynél elesites elott tovaabbi vedelmi lepesek kellenek.

10. Teszteles es validacio

A teszteles celja annak bizonyitasa, hogy a fo felhasznaloi es adminisztracios folyamatok nem csak elmeletben leteznek, hanem ellenorizhetően mukodnek. A TDLWebshop eseteben a teszteles ket szinten tortent: automata tesztekkel es kezi tesztjegyzokonyvvel.

Az automata tesztek elonye, hogy a CI folyamatban is futtathatok, így egy GitHub Actions zöld állapot kulso bizonyitek a build es tesztek sikeressegere. A kezi teszteles ezzel szemben a teljes felhasznaloi elmenyt, a kepernyoallapotokat, hibauzeneteket es reszponzivitast vizsgalja.

A dolgozatban kulonosen fontos megmutatni, hogy a kritikus folyamatok ellenorizve voltak: checkout, rendelés létrejött, készletváltozás, admin státuszváltás, tiltott jogosultság, kuponvalidacio, CSV import es AI asszisztens.

10. tablazat - Tesztelesi terv osszefoglalasa

Tesztazonosito	Folyamat	Elvart eredmény	Bizonyitek
T1	Regisztracio es belepés	Felhasznalo profilja létrejön, tiltott user üzenetet kap.	Kezi teszt
T2	Termekkeresés	A lista szűrodik, nincs talalat eseten üres állapot látszik.	Kezi teszt
T3	Kosár mennyiség	Osszeg frissül, törles mukodik.	Automata/kezi
T4	Checkout validacio	Hibas email/telefon nem enged tovább.	Kepernyokep
T5	Sikeres rendelés	Rendelés létrejön Firestore-ban es adminban látszik.	Kezi teszt
T6	Admin státuszváltás	Audit es készletváltozás rögzodik.	Automata/kezi
T7	CSV import	Hibas sorok jelzodnek, ervenyes sorok menthetőek.	Kezi teszt
T8	PDF bizonylat	PDF tartalmazza az	Kepernyokep

		adatokat es nem csuszik össze.	
T9	AI asszisztens	Relevans kerdesre domainvalasz, irrelevansra elhatarolas.	Kezi teszt
T10	CI	GitHub Actions build/test zold.	Kepernyokep

10.1. Build es CI

A projekt CI folyamata GitHub Actions segitsegevel fut. A workflow celja, hogy friss commit utan ellenorizze a telepithetoseget, buildet es teszteket. A zold CI allapot a dolgozatban kulon kepernyokepkent szerepeljen, mert gyorsan bizonyitja, hogy a repo nem csak lokalis gepen mukodik.

[KEP / ABRA HELYE] GitHub Actions zold CI futas

Beillesztendo tartalom: GitHub Actions oldalon a legfrissebb zold workflow run latszodjon, commit nevvvel es idotartammal.

10.2. Kezi tesztjegyzokonyv

A kezi tesztjegyzokonyvet a dolgozat mellekleteben vagy a teszteles fejezetben erdemes bemutatni. A teszteles soran nem csak a sikeres utakat, hanem a hibaagakat is ellenorizni kell. Ilyen pelda a hibas email, tiltott felhasznalo, keszlethiany, jogosulatlan admin muvelet vagy nem relevans AI kerdes.

[KEP / ABRA HELYE] Checkout validacios hiba

Beillesztendo tartalom: Checkout oldal hibas email vagy telefonszam beirasaval.

A vasarloi ut reszletes ertelmezese

A vasarloi ut a kezdolapon indul, ahol a felhasznalo gyorsan kepet kap a webshop temajarol es a fobb kategoriakrol. A keresosav kiemelt szerepet kap, mert egy epulelgepeszeti webshopban a vasarlo sokszor konkret termeknevet, meretet vagy cikkszamat keres. A kategoriak dropdown formaja egyszerre ad gyors navigaciot es modern feluleti elmenyt.

A termeklista celja az, hogy a vasarlo ossze tudja hasonlítani a kinalatot. A termekkartyakon az ar, keszlet es termeknev egyutt jelenik meg. A kedvezmenyes vagy uj

termékek jelölése segíti a döntést, de a felület nem viszi túlzásba a vizuális elemeket, mert a muszázi termékeknel az olvashatóság fontosabb, mint a pusztán reklám jellegű megjelenés.

A kosár oldal a döntés és a rendelés közötti átmenet. Itt a felhasználó ellenőrizheti a mennyiséget, törölhet tételt, majd továbbléphet a checkoutra. A kosár logikája azért fontos, mert az összegzésnek mindig követnie kell a mennyiségváltozást, és nem szabad instabil termékazonosítókra épülnie.

A végleges dolgozatban ezt a részt érdemes saját tapasztalatokkal kiegészíteni: milyen hibát találtál, mit javítottál, milyen döntést hoztál, és a védesen hogyan tudnád saját szavaiddal elmagyarázni.

Adminisztrációs folyamatok részletes értelmezése

Az admin felület nem egyszerű másolata a vásárlói oldalnak. Más célcsoportot szolgál ki, ezért sürűbb információk elrendezést, gyors műveleti gombokat és több adatot tartalmaz. Egy adminnak nem marketinges termékbemutatót kell látnia, hanem olyan adatokat, amelyek alapján rendelést, készletet és vásárlót tud kezelni.

A helyszíni vásárlás funkció különösen fontos a projektben, mert egy valószínűleg épületegészítési boltban a rendelések nem mindig weben érkeznek. Az adminnak vagy dolgozonak gyorsan ki kell tudnia választani egy mentett vásárlót, terméket kell keresnie, mennyiséget kell megadnia, majd bizonylatot kell generálnia. Ez a funkció emiatt a domainhez illeszkedő, nem általános webshopos kiegészítés.

A dolgozói jogosultságok bevezetése azt mutatja, hogy a rendszer nem egyetlen admin felhasználóra épül. A dolgozó tud terméket és készletet kezelni, vásárlást rögzíthet, de nem kap minden jogosultságot. Ez közelebb viszi a rendszert egy valószínű szervezeti működéshez.

A végleges dolgozatban ezt a részt érdemes saját tapasztalatokkal kiegészíteni: milyen hibát találtál, mit javítottál, milyen döntést hoztál, és a védesen hogyan tudnád saját szavaiddal elmagyarázni.

Adatbiztonsági és minőségi szempontok részletes értelmezése

A Firestore szabályokkal kapcsolatban fontos felismerés, hogy a kliensoldali ellenőrzés csak felhasználói kényelmi és UX szempontból elég. Biztonsági szempontból az adatbázis oldali szabályok jelentik az igazi védelmi vonalat. Emiatt a szerepkörök, aktív felhasználói állapot és payload mezők ellenőrzése szakdolgozati szinten is értékes része a projektnek.

A titkok kezelése külön fejezetet érdemel, mert modern webes fejlesztéssel az API kulcsok és jelszavak véletlen commitolása gyakori hiba. A projektben a végleges irány az, hogy a repóban csak példa konfiguráció szerepel, az éles kulcsok pedig szolgáltatói secretben tárolodnak. Ez mernoki és biztonsági szempontból is helyes irány.

A tesztelés és CI nem csak technikai formalitás. A zöld CI azt bizonyítja, hogy a projekt egy külső futtatókörnyezetben is le tud fordulni és a tesztek lefutnak. Ez különösen fontos a konzulensi visszajelzés alapján, mert a repo rendezettség és reprodukálhatósága a végleges értekezésben is szerepet kap.

A végleges dolgozatban ezt a részt érdemes saját tapasztalatokkal kiegészíteni: milyen hibát találtál, mit javítottál, milyen döntést hoztál, és a védesen hogyan tudnád saját szavaiddal elmagyarázni.

Továbbfejlesztési lehetőségek részletes értelmezése

Éles környezetben a legfontosabb továbbfejlesztés a szerveroldali üzleti logika erősítése lenne. A webes rendelesek arat, kedvezményét és készletet nem ideális teljes mértékben kliensoldali adatokra bízni. Egy Cloud Function vagy külön backend endpoint újraszámolhatná a végösszeget és ellenőrizhetné a készletet.

Az AI asszisztens továbbfejlesztéséhez hasznos lenne keresési index, termékleírásokból készült strukturált tudástár és rate limit. Ez csökkentene a pontatlan ajánlások és visszajelzések kockázatát. A rendszer jelenlegi állapotában prototípusnak tekinthető, amely megmutatja az irányt, de nem helyettesíti a szakmai ügyfélszolgálatot.

A készletfigyelés tovább bővíthető fogási trenddel, utánrendelési javaslattal és admin értesítésekkel. Ez különösen jól illeszkedik az épületgépészeti kereskedelemhez, ahol bizonyos szerelési anyagok és gyakran használt alkatrészek készleten tartása üzletileg fontos.

A végleges dolgozatban ezt a részt érdemes saját tapasztalatokkal kiegészíteni: milyen hibát találtál, mit javítottál, milyen döntést hoztál, és a védesen hogyan tudnád saját szavaiddal elmagyarázni.

11. Mernoki döntések és részletes szakmai indoklás

Az alábbi alfejezetek azt a célt szolgálják, hogy a dolgozat ne csak rövid fejezetváz legyen, hanem a konzulensi elvárásoknak megfelelően részletesen is bemutassa a rendszer szakmai

hatteret, döntéseit, korlátait és bizonyítékait. A végleges változatban ezeket a részeket saját megfogalmazásra kell húzni, de tartalmilag a TDLWebshop mernoki történetét követik.

A projekt valós felhasználói problémája

A TDLWebshop alap gondolata nem pusztán az volt, hogy készüljön egy termékeket listázó weboldal. Az épütelgépészeti kereskedelemben a vásárlói és üzemeltetői folyamatok egyszerre jelennek meg: a vásárló terméket keres, rendelést ad le, kedvezményt használ, a dolgozó pedig készletet ellenőriz, rendelést rögzít és bizonylatot készít. Ez a kettős igény adta a projekt szakdolgozati értékét, mert a rendszernek nem egyetlen oldalt, hanem több, egymással összefüggő munkafolyamatot kellett kezelnie.

A dolgozatban ezt azért fontos részletesen bemutatni, mert egy webshop csak akkor tekinthető termékszerű MVP-nek, ha a felhasználó nem akad el a kulcsfolyamatokban. A kezdőlap, terméklista, kosár és checkout önmagában még nem elég, ha nincs mellette adminisztráció, rendeltetés-történet, jogosultságkezelés és készletlogika. A TDLWebshop ezek közül több területet is összekapcsol, ezért a projekt közelebb áll egy valós üzleti alkalmazáshoz, mint egy egyszerű tanuloprojekthez.

A valós probléma másik oldala az, hogy az épütelgépészeti termékeknel a vásárló sokszor nem csak nevet vagy árat néz. Fontos lehet a kategória, a műszaki jellemző, a készlet, a szállítási lehetőség és az, hogy helyszíni vásárlás során is rögzíthetők legyenek a tételtek. Emiatt a rendszer nem csak látványos felületet kapott, hanem olyan admin oldali funkciókat is, amelyekkel a bolt munkafolyamatai bizonyíthatók.

A végleges dolgozatban ide érdemes beilleszteni egy rövid saját történetet arról, miért pont épütelgépészeti webshop lett a téma. Ez segít abban, hogy a dolgozat ne általános szoftverleírásnak hasson, hanem látszódjon mögötte a személyes motiváció és a domain ismerete.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Az MVP határának indoklása

A szakdolgozati MVP határa tudatosan úgy lett kijelölve, hogy a rendszer a vásárlói utat és az adminisztrációs utat egyaránt bizonyítsa. Nem célja a projektnek, hogy teljes értékű vállalatirányítási rendszer, bankkártya-elfogadó rendszer vagy számlázó program legyen.

A cél az, hogy a rendelestől a készlet- és admin kezelésig bemutatható legyen egy működő, bővíthető tervezett webshop alap.

Ez a határ azért védhető, mert a szakdolgozat nem egy éles kereskedelmi szolgáltatás teljes jogi és pénzügyi megfelelést vállalja, hanem egy mérnoki indokolt MVP-t. A bankkártya fizetés például nem teljes online fizetési integráció, hanem a fizetési módok modellezése és a rendelési folyamat része. A PDF bizonylat sem hivatalos e-számla szolgáltatás, hanem a rendelet összefoglaló dokumentum, amely bemutatja a generált kimenet logikáját.

Az MVP-határ része az is, hogy bizonyos kockázatokat a dolgozat korlátként nevez meg. Ide tartozik a webes rendelések szerveroldali végösszeg-újraszámítása, az AI asszisztens pontos termékajánlasi korlátja, valamint az éles fizetési szolgáltató hiánya. Ezek nem elhallgató hibák, hanem olyan továbbfejlesztési pontok, amelyek egy szakdolgozatban kifejezetten jól mutatják a realis onertékelést.

A bírálo számára különösen hasznos, ha az MVP-határ táblázatban is megjelenik: mi készült el, mi maradt prototípus, és mi tudatosan nem lett bevállalva. Ezzel elkerülhető, hogy a projekt felkészülésnek hasson, miközben valójában meghatározott célra készült.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszt vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Piaci összehasonlítás szerepe

A piaci összehasonlítás célja nem az, hogy a TDL Webshop minden nagy webáruháznál jobb legyen. A cél az, hogy látszódjon: milyen megoldások jellemzők a hasonló webshopokra, és ehhez képest a saját rendszer milyen szakdolgozati többletet ad. A konzulensi visszajelzés alapján ez a fejezet különösen fontos, mert megmutatja, hogy a projekt nem elszigetelt kódgyakorlat.

Az összehasonlításban érdemes 2-4 épülelgepeszeti vagy muszaki webshopot vizsgálni. A szempontok lehetnek: termékkeresés, kategória struktúra, kosár, checkout, regisztrált profil, admin jellegű funkciók láthatósága, készletinformáció, szakmai támogatás, akciós termékek és mobilos használhatóság. A saját rendszer értéke nem feltétlenül abban van, hogy több terméket tartalmaz, hanem abban, hogy a vásárlói és admin oldali funkciókat egy szakdolgozati MVP-ben összekapcsolja.

A TDLWebshop egyik erősebb pontja a helyszini vásárlás rögzítése, a mentett vásárlók kezelése és a dolgozói szerepkör megjelenése. Ezek olyan funkciók, amelyek sok publikus webshopon nem láthatók, mert belső vállalati felületekhez tartoznak. Szakdolgozati szempontból viszont pont ezek bizonyítják, hogy a rendszer nem csak kirakatoldal.

A piaci fejezetet a végleges dolgozatban egy táblázattal érdemes zárni. A táblázatban ne csak pipákat használj, hanem rövid megjegyzéseket is: miért fontos az adott szempont, és a TDLWebshop hogyan kezeli.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszt vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Követelmények és traceability

A követelmények fejezete köt össze a célkitűzés és a megvalósítás között. Ha egy rendszerben sok funkció van, könnyen elveszik, hogy melyik mire szolgált. A traceability táblázat ezt oldja meg: követelmény, use case, megvalósított modul és teszt bizonyíték egy sorban látszik.

A TDLWebshop esetében külön kell kezelni a vásárlói követelményeket, az adminisztrációs követelményeket és a biztonsági követelményeket. A vásárlói oldalhoz tartozik a termékkeresés, kosár, checkout, profil és rendeléstörténet. Az admin oldalhoz tartozik a termékfeltöltés, CSV import, rendeléskezelés, készletfigyelés, helyszini vásárlás, mentett vásárlók és jogosultságkezelés.

A biztonsági követelmények nem csak technikai részletek. Ide tartozik, hogy tiltott felhasználó ne tudjon tovább rendelni, normál vásárló ne érjen el admin funkciókat, a dolgozó csak a saját szerepkörének megfelelő felületet lássa, és API kulcs ne szerepeljen a repóban. Ezeket a dolgozatban külön érdemes kifejteni, mert a webes rendszereknél gyakori bírálati pont a jogosultságkezelés.

A traceability táblázat azért is jó, mert a tesztelés fejezetéhez is hidat ad. Ha leírod, hogy például a checkout validációhoz melyik képernyőkép és melyik teszt tartozik, akkor a bíráló gyorsan látja, hogy nem csak állítod a működést, hanem bizonyítod is.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszt vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

GUI/UX döntések indoklása

A felület tervezésénél a cél egy modern, ipari hangulatu webshop volt. A dark mode, a kek-piros accent színek és a TDLWebshop logo világa együtt adja azt a vizuális irányt, amely az épületegészeti témához illik. A fontos döntés az volt, hogy a látvány ne menjen az olvashatóság rovására.

A kezdőlap szerepe az első benyomás. Itt a kategória dropdown, a keresősáv, a hero szekció és a kiemelt termékek vezetik a felhasználót. A terméklista és termékkártya részeknél már a funkcionalitás erősebb: terméknev, ár, készlet, kosárba helyezés és részletek gomb. A dolgozatban a képernyőképeken keresztül érdemes megmutatni, hogy ugyanaz a vizuális rendszer több oldalon is visszatér.

A light mode külön kihívás volt, mert nem elég a háttérre fehérre és a szöveget feketére váltani. A kontraszt, gombok, fejléc és kártyák árnyékolása külön figyelmet igényel. Ezt a dolgozatban UX döntésként lehet leírni: a layout azonos marad, de a színek és árnyékok alkalmazkodnak a témához.

Az admin felületnél a design más szempontot követ. Itt a sürűbb információ, a gyors gombok, táblázatok, filterek és státuszok a fontosak. A dolgozatban jó, ha külön képernyőkép van a vásárlói oldalról és az admin oldalról, mert így látszik, hogy két célcsoporthoz más felületi logika tartozik.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kód részlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Angular komponensalapú felepítés

Az Angular használata azért indokolt, mert a rendszer több oldalból, szolgáltatásból és állapotból áll. A komponensek és service-ek elkülönítése segít abban, hogy a megjelenítés és az adatkezelés ne keveredjen teljesen össze. Egy webshopnál ez különösen fontos, mert ugyanazt a termékadatot több helyen is használni kell.

A terméklista, kosár, checkout, profil és admin oldalak külön felelőségeket kapnak. A service retegek, például a product, cart, order, auth vagy chatbot service, az adatbetöltés és üzleti logika közelebbi részeit kezelik. Ez a felepítés a dolgozat architektúra fejezetében jól bemutatható, mert látszik belőle a kliensoldali alkalmazás szerkezete.

A komponensalapú felbontás másik előnye a tesztelhetőség. Bizonyos részek önállóan vizsgálhatók, például a kosárösszegezés, kuponlogika vagy validáció. A szakdolgozatban nem kell minden tesztet részletesen bemutatni, de a kritikus folyamatokhoz érdemes egy-egy példát hozni.

A végleges szövegben érdemes röviden kitérni arra is, hogy az Angular választása milyen alternatívákhoz képest történt. Például React vagy Vue is alkalmas lett volna, de az Angular strukturaltsága és service-orientált felépítése illeszkedett a nagyobb alkalmazás jellegű projekthez.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódrészlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Firestore és Firebase szerepe

A Firebase a projektben backend-szerű szolgáltatásokat ad: hitelesítés, Firestore adatbázis, hosting és biztonsági szabályok. Ez lehetővé tette, hogy a szakdolgozat fókuszba ne teljesen egy saját backend infrastruktúrára menjen el, hanem a webshop üzleti folyamataira és adatmodelljére.

A Firestore dokumentumalapú modellje jól illeszkedik a termékek, rendelések, felhasználói profilok, kuponok és mentett vásárlók tárolásához. Ugyanakkor a dokumentumalapú adatbázis nem ugyanazt a gondolkodást igényli, mint egy relációs adatbázis. A dolgozatban érdemes megmutatni, mely entitások külön collectionbe kerültek, és milyen adatok kapcsolják őket össze.

A biztonsági szabályok külön fejezetet érdemelnek. A kliensoldali kód önmagában nem véd elegendő, mert egy rosszindulatú felhasználó megpróbálhat közvetlenül adatbázishoz fordulni. Emiatt a Firestore rules oldalon is kezelni kell, ki mit olvashat és írhat. Ez a projekt egyik erősebb mérnöki pontja.

A Firebase config és API kulcs kérdése külön magyarázatot igényel. Webes Firebase alkalmazásoknál a kliensoldali config részben publikus, de ettől még a valódi védelem a security rules és a jogosultsági modell. A dolgozatban érdemes leírni, hogy titkok, OpenRouter kulcsok és jelszavak nem kerülhetnek repóba.

Kapcsolodo bizonyitek a vegleges dolgozatban: ide kerulhet kepnyokep, rovid kodreszlet vagy tablázat, amely az adott allitast alatasztja. A cel, hogy a szoveg ne onmagaban alljon, hanem lathato rendszerreszhez, teszthez vagy dokumentalt donteshez kapcsolodjon.

Rendeleskezeles es keszletlogika

A rendeleskezeles a webshop egyik legfontosabb folyamata. A vasarloi oldalon a checkout gyujti ossze a kosarat, szallitasi adatokat, fizetesi modot es kedvezmenyt. Az admin oldalon a rendeles statusza kovetheto, modositlato, es a keszletvaltozas is kapcsolodik hozza.

A helyszini vasarlasnal kulonosen fontos volt, hogy a rendeles, keszletvaltozas es bizonylat ne teljesen kulon mozogjon. Ha egy admin rogzit egy eladast, a rendszernek figyelembe kell vennie a kivalasztott termekeket, mennyisegeket es vasarlot. Ez a folyamat szakdolgozati szempontbol ertekesebb, mint egy egyszeru kosarmentes, mert belso bolti munkafolyamatot modellez.

A webes rendelesehez kapcsolodo korlatot oszinten erdemes leirni. Ha bizonyos arak es kedvezmenyek kliensoldalrol jonnek, akkor eles rendszerben szerveroldali ellenorzesre lenne szukseg. Ez nem feltetlenul gyengiti a szakdolgozatot, ha a korlat vilagosan szerepel es tovaabfejlesztési terv is tartozik hozza.

A statuszvaltas es audit naplo azt mutatja, hogy a rendeles állapotvaltozasai kovethetok. Ez a dolgozatban jo pelda arra, hogyan lehet egy uzleti folyamatot nem csak vegeteredmenykent, hanem tortenettel együtt kezelni.

Kapcsolodo bizonyitek a vegleges dolgozatban: ide kerulhet kepnyokep, rovid kodreszlet vagy tablázat, amely az adott allitast alatasztja. A cel, hogy a szoveg ne onmagaban alljon, hanem lathato rendszerreszhez, teszthez vagy dokumentalt donteshez kapcsolodjon.

PDF bizonylat es szamlageneralas

A PDF generalas a projektben a rendelési folyamat kezzelfoghato kimenete. A vasarlo vagy admin nem csak adatbazis-bejegyzest kap, hanem letoltheto dokumentumot is. Ez szakdolgozati szempontbol jól demonstralja, hogy a rendszer képes strukturalt adatbol dokumentumot eloallitani.

A bizonylat felepitese tartalmazza a rendeles azonositojat, vevoi adatokat, termekeket, fizetési es szallitasi modot, valamint a vegosszeget. A korabbi visszajelzes alapján fontos

volt az elrendezés javítása, hogy az összeg ne csusszon bele más blokkokba. Ezt a dolgozatban akár fejlesztési tapasztalatként is meg lehet említeni.

Eles környezetben egy jogilag érvényes e-számla más követelményeket jelentene. Ehhez külső számlázó szolgáltatás, jogszabályi megfelelés és adatszolgáltatási logika kellene. A TDLWebshop jelen állapotában PDF bizonylatot generál, ami MVP szinten bemutatja a dokumentumkészítés elvét.

A végleges dolgozatban ide mindenképpen kell egy kép a generált PDF-ről. A kép mellett röviden magyarázni kell, mely adatok a rendelesekből származnak, és melyek fix céladatok vagy számlaformázási elemek.

Kapcsolódó bizonyítékok a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódrészlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

AI asszisztens korlátai és szerepe

Az AI asszisztens nem azért került a rendszerbe, hogy szakembert vagy ügyfélszolgálatot teljesen helyettesítsen. A cél az volt, hogy a termékkatalogushoz és épületegészítési témákhoz kapcsolódóan segítse a tájékozódást. Ez fontos különbség, mert egy szakdolgozati prototípusban az AI-használatot korlátokkal együtt kell bemutatni.

A rendszerben az OpenRouter hívás nem közvetlenül a kliensből történik, hanem Cloudflare Worker proxy-n keresztül. Ez azért fontos, mert az API kulcs nem kerülhet a böngészőbe vagy a repóba. A dolgozatban ez biztonsági és architektúrai döntésként is szerepelhet.

A válaszoknál külön szabály, hogy az asszisztens ne ajánljon random terméket akkor, ha nincs megfelelő katalogus-talalat. Ilyenkor adhat általános szakmai irányt, de jeleznie kell, hogy pontos ajánlatért vagy beszerezhetőseget emailben vagy személyesen érdemes egyeztetni. Ez csökkenti a téves termékajánlás kockázatát.

A dolgozatban külön kell választani a fejlesztés során használt AI-t és a webshopba beépített AI asszisztensét. Az előbbi munkaeszköz volt, az utóbbi a kész rendszer egyik funkciója. Ez a különválasztás megfelel a konzulensi elvárásnak is.

Kapcsolódó bizonyítékok a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódrészlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Tesztelesi bizonyítás és kezi ellenőrzés

A teszteles fejezet célja, hogy a rendszer működését ne csak állítások támasszák alá. A build, az automata tesztek és a kezi tesztjegyzék egyaránt adnak bizonyítékot. Ez különösen fontos, mert a szakdolgozatban több felhasználói szerepkör és több kritikus folyamat szerepel.

Az automata tesztek közül érdemes kiemelni a kosár, checkout/kuponlogika, admin státuszváltás, PDF generálás, AI asszisztens és validáció tesztjeit. Nem kell minden assertet bemutatni, de a dolgozatban legyen egy táblázat: tesztelt funkció, módszertan, várt eredmény, tényleges eredmény.

A kezi tesztek a vizuális és folyamatbeli részek miatt szükségesek. Ilyen példa a mobilos megjelenés, admin táblázatok kezelhetősége, checkout hibajelzések, mentett vásárló kiválasztása vagy PDF letöltés. Ezeket screenshotokkal és pipálható tesztlistával lehet igazolni.

A GitHub Actions zöld CI képernyőképe erős bizonyíték, mert megmutatja, hogy a projekt nem csak a saját gépen futott. A végleges dolgozatban ezt a reprodukálhatósági fejezethez érdemes tenni.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódcsúszka vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Repo-higiénia és beadáshoz tartozó tisztaság

A konzulensi visszajelzés egyik fő pontja a repo rendezettsége volt. Ez nem mellékes adminisztráció, hanem a szakdolgozat bírálatának feltétele. Ha a repóban `node_modules`, build mappák, jelszavak vagy helyi segédfájlok vannak, akkor a projekt kevesbé professzionálisnak tűnik.

A végleges repóban a futtatható kód, konfigurációs minták, dokumentációnak és teszteknek kell szerepelniük. A személyes segédanyagok, hosszú átirati alapok és munkapeldányok lokálisan maradhatnak, de nem kell őket GitHubra feltölteni. Ez különösen fontos, mert a dolgozat saját nyelvezetre huzása a hallgató feladata.

A `.env.example` szerepe az, hogy megmutassa, milyen környezeti változókra van szükség, de ne tartalmazzon valódi kulcsot. Az OpenRouter API kulcsot a Worker secret kezeli, nem a repóban tárolt fájl. Ez a titokkezelésről szóló részen jó példa.

A README-nek olyan személynek is segítenie kell, aki nem vett részt a fejlesztésben. Telepítés, indítás, build, teszt, Firebase, Worker és demo szerepkörök: ezek legyenek egyértelműek. A dolgozat reprodukálhatósági fejezete erre hivatkozhat.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Sajat értékelés és tanulságok

A dolgozat végi reflexió nem egyszerű lezáró bekezdés. Itt kell megmutatni, hogy mit tanultál a projektből, milyen döntések voltak nehezek, és hol látod a rendszer korlátait. A jó szakdolgozat nem hibamentesnek mutatja a projektet, hanem realisan értékeli.

Érdekes tanulság lehet, hogy a webshop fejlesztése közben a látványos felület csak az egyik rész. Legalább ilyen fontos lett a jogosultságkezelés, Firestore szabályok, tesztek, adatmodell és dokumentáció. Ez jól mutatja, hogy egy szoftveres projektben a kód mellett a bizonyíthatóság is mérnöki munka.

A nehézségek közé bekerülhet a checkout validáció, a PDF elrendezés, a helyszíni vásárlás mentése, a CSV import, az AI proxy biztonságos bekötése vagy a GitHub CI stabilizálása. Ezek konkrét, védesen is jól elmondható fejlesztési tapasztalatok.

A továbbfejlesztési irányok között reális marad a szerveroldali rendelésellenőrzés, éves fizetési szolgáltató, fejlettebb készlet-előrejelzés, admin riportok és AI asszisztens rate limit. Ezeket nem kell most mind megvalósítani, de jól zárják a dolgozatot.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Bizonyíték	Hova kerüljön	Mit igazol
GitHub Actions zöld CI képernyőkép	Reprodukálhatóság és tesztelés fejezet	A projekt tiszta környezetben is buildelhető és tesztelhető.
Firestore rules kódreszlet	Biztonsági fejezet	A jogosultságok nem csak kliensoldalon, hanem adatbázis szinten is kezeltek.
Checkout validáció képernyőkép	GUI/UX és tesztelés fejezet	A hibás bemeneteket a rendszer felhasználói szinten is kezeli.

Helyszini vasarlas kepernyokep	Megvalositas fejezet	Az admin oldali belso bolti folyamat is mukodik.
PDF bizonylat kepernyokep	Megvalositas es eredmenyek fejezet	A rendelési adatbol strukturalt dokumentum keszul.
AI asszisztens kepernyokep	MI-hasznalat es megvalositas fejezet	A rendszerben mukodo AI funkcio elkulonul a fejlesztet segito AI-hasznalattol.

12. Reprodukálhatóság, CI és deploy

A konzulensi visszajelzés alapján a reprodukálhatóság kulcsfontosságú. Egy szakdolgozati repo akkor bírható nyugodtan, ha egy másik környezetben is elindítható, és a szükséges lépések nem csak a fejlesztő saját gépen ismertek. Emiatt a README, .env.example, telepítési leírás, demo szerepkörök és CI bizonyíték kiemelt szerepet kapnak.

A repo nem tartalmazhat node_modules, build mappákat, valódi jelszavakat, API kulcsokat vagy gépfuggó állományokat. A függőségek package.json és lock file alapján telepíthetők, a titkok pedig külön környezeti változókban vagy szolgáltatói secretként tárolandók.

A deploy két külön részből állhat: a frontend Firebase Hostingra kerül, az AI asszisztens szerveroldali proxyja pedig Cloudflare Workerként működik. A dolgozatban ezt a szétválasztást érdemes röviden elmagyarázni, mert jól mutatja a kulcsvédelem és a kliens/szerver felelősségi határainak megértését.

11. táblázat - Reprodukciós és deploy lépések

Lépés	Parancs / ellenőrzés	Cél
Függőségek telepítése	npm install	Angular projekt függőségeinek telepítése.
Build	npm run build	Frontend fordíthatóságának ellenőrzése.
Teszt	npm test -- --watch=false	Automata tesztek futtatása.
Hosting deploy	firebase deploy --only hosting	Frontend publikálása.
Firestore rules deploy	firebase deploy --only firestore:rules	Biztonsági szabályok elesítése.
Worker deploy	npx wrangler deploy	OpenRouter proxy publikálása.

13. Mesterseges intelligencia hasznalata a fejlesztés során

A szakdolgozat készítése során mesterséges intelligencia több ponton segítette a munkát. Fontos különválasztani két területet: az egyik a fejlesztési folyamatban használt AI-támogatás, a másik pedig maga a TDLWebshopban megjelenő vásárlói AI asszisztens. A kettő nem ugyanaz: az előbbi fejlesztői segédeszköz, az utóbbi a rendszer egyik funkciója.

A fejlesztés során AI-támogatás elsősorban ötletelesre, kódreview-ra, hibakeresésre, dokumentációs szerkezet kialakítására, tesztforgatókönyvek összeállítására és nyelvi javításokra lett használva. A kimenetek nem automatikusan kerültek elfogadásra: a fejlesztő feladata maradt a kód megértése, ellenőrzése, futtatása és a döntések meghozatala.

Az AI használata során külön figyelmet kellett fordítani a titkok kezelésére. API kulcsot vagy jelszót nem szabad a kódba vagy nyilvános repóba helyezni. Egy korábbi kulcsot vissza kellett vonni, majd a véglegesebb megoldásban az OpenRouter kulcs Cloudflare Worker secretként került kezelésre.

Az AI korlátai a fejlesztésben is megjelentek. Előfordulhat, hogy egy javaslat elavult, nem illeszkedik a projekt szerkezetébe, vagy túl általános megfogalmazást ad. Emiatt a végleges szakdolgozati szöveget saját nyelvezetre kell átírni, és minden állítást a működő rendszerrel, tesztekkel vagy dokumentációval kell alátámasztani.

A rendszerben működő AI asszisztens szinten korlátos. Nem célja, hogy szakmERNOKI felelősséggel teljes műszaki ajánlatot adjon. A cél inkább az, hogy a termékkatalógus alapján irányt mutasson, és ha nincs pontos találat vagy biztos adat, akkor javasolja az email-es vagy személyes egyeztetést.

12. táblázat - AI-használat és ellenőrzés

AI-használat területe	Mire szolgált	Validáció
Ötleteles	Funkciók, MVP-határ és dokumentációs fejezetek átgondolása.	Konzulensi elvárásokkal összevetve.
Kódreview	Hibalehetőségek, jogosultságok, validáció és tranzakciók vizsgálata.	Kód olvasása, build és teszt.
Dokumentáció	Fejezetváz, tesztjegyzék és biztonsági minimum	Saját átirás és szakmai ellenőrzés.

	elokeszítése.	
AI asszisztens funkció	Felhasználói kérdések katalógushoz kötött megválaszolása.	Domainkérdés, termékkérdés és irreleváns kérdés tesztje.

14. Összefoglalás, korlatok és továbbfejlesztés

A TDLWebshop egy épütelgépészeti témájú webshop és adminisztrációs rendszer MVP-je. A projekt bemutatja, hogyan épülhet fel egy vásárlói oldalból, admin felületből, Firestore adatmodellből, jogosultsági szabályokból, PDF bizonylatból, CSV importból, készletkezelésből és AI asszisztensből álló webes rendszer.

A munka legerősebb része a funkciók összekapcsolása: a vásárlói rendelés nem önmagában áll, hanem megjelenik az admin felületen, státuszt kap, PDF bizonylattal és készletlogikával kapcsolódik a belső folyamatokhoz. A helyszíni vásárlás és mentett vásárló kezelése külön domaintereket ad a rendszernek.

A rendszernek vannak tudatos korlátai. Ilyen a teljes pénzügyi integráció hiánya, a webes rendelések szerveroldali arújrásamolásának továbbfejlesztési igénye, a készletfoglalás ipari szintű kezelése és az AI asszisztens rate limit/kvóta oldali erősítésének szükségessége. Ezeket nem hanyagosságment, hanem továbbfejlesztési irányként kell bemutatni.

[SAJÁT ZÁRO REFLEXIO HELYE] Ide 1-2 bekezdésben saját hangon írd le, mit tanultál a projektből, mi volt a legnehezebb, mit csinálnál másként, és melyik részere vagy a legbuszkebb. Ez legyen teljesen saját megfogalmazás, mert a védesen is ebből tudsz a legtermészetesebben beszélni.

Irodalomjegyzék

Angular dokumentáció - <https://angular.dev/>

Firebase dokumentáció - <https://firebase.google.com/docs>

Cloud Firestore Security Rules dokumentáció - <https://firebase.google.com/docs/firestore/security/get-started>

Cloudflare Workers dokumentáció - <https://developers.cloudflare.com/workers/>

OpenRouter dokumentáció - <https://openrouter.ai/docs>

GitHub Actions dokumentacio - <https://docs.github.com/actions>

[További piaci összehasonlításban használt webshopok forrásai]

[Intézményi szakdolgozati formai és tartalmi útmutató]

Mellekletek

M1. Kezi tesztjegyzék könyv kitöltött állapotban.

M2. Kiemelt kódrészletek: checkout, OrderService, Firestore rules, InvoiceService, AI proxy.

M3. Képernyőképek gyűjteménye: vásárlói oldal, admin oldal, AI asszisztens, CI.

M4. Reprodukciós lépések és környezeti változók mintája.